

NAME: Sai Tharun Venkatramanan
ASU ID: 1234802844

CSE 539 PROJECT: TLS Handshake with PAKE

1 Objective:

This project implements a simplified TLS 1.3 style handshake integrated with password based client authentication. In normal TLS, the server is authenticated using certificates, but the client is often authenticated later by sending a password inside the already established encrypted channel. This project instead integrates password authentication into the handshake itself.

The implemented system allows a client and a server to establish a secure channel over localhost. The server is authenticated using a CA signed certificate and a digital signature over the handshake transcript. The client is authenticated using a password derived authenticator inspired by PAKE style authentication. After mutual authentication, both parties derive application traffic keys using HKDF and exchange encrypted application messages using AES-GCM.

2 System Architecture:

The system consists of three logical parties: Certificate Authority, Server, Client.

2.1 Certificate Authority

The Certificate Authority is responsible for creating and signing the server certificate. In this implementation, the CA is represented by the `ca.py` script rather than a continuously running network service. The CA generates an Ed25519 signing key pair. It also generates the server signing key pair and creates a server certificate containing the server public key. The certificate is signed using the CA private key. The client later uses the CA public key to verify this certificate.

The CA produces the following files:

- `certs/ca_private.pem`
- `certs/ca_public.pem`
- `certs/server_private.pem`
- `certs/server_public.pem`
- `certs/server_cert.json`

2.2 Server

The server runs as a separate process and listens on `127.0.0.1:4444`. It stores its private signing key, its CA signed certificate, and a password database containing registered users. It receives the ClientHello message, and then sends a ServerHello message containing its certificate, ephemeral public key, password salt, and signature. Followed by verification of the client's password based authenticator and sending a ServerFinished message if client authentication succeeds. It derives application traffic keys, decrypts the client's encrypted application message and then sends an encrypted response.

2.3 Client

The client also runs as a separate process. It connects to the server over localhost and initiates the handshake. The client stores the CA public key and uses it to verify the server certificate. The client also takes a username and password as command line input. During the handshake, the client derives a password based value from the password and uses it to compute an authenticator.

The client performs the following functions:

1. Sends a ClientHello message.
2. Verifies the server certificate.
3. Verifies the server signature.
4. Computes the password derived verifier.
5. Sends a ClientAuth message.
6. Verifies the ServerFinished message.
7. Derives application traffic keys.
8. Sends and receives encrypted application data.

2.4 Protocol Design

2.4.1 ClientHello

The client begins by sending a ClientHello message. This message includes the username, a fresh client nonce, and the client's ephemeral X25519 public key. The client nonce and ephemeral public key provide freshness and contribute to the key exchange.

2.4.2 ServerHello

The server responds with a ServerHello message. This message includes the server nonce, server ephemeral X25519 public key, server certificate, password salt, and server signature. The server signature is computed over the handshake transcript. The client verifies this signature using the server public key from the verified certificate.

2.4.3 ClientAuth

After receiving the password salt, the client derives a password verifier from the password. The client then combines this password derived value with the X25519 shared secret to derive the handshake secret. The client sends an HMAC authenticator in the ClientAuth message. If the client used the wrong password, the derived handshake secret will be different, and the server will reject the authenticator.

2.4.4 ServerFinished

If the client authenticator is valid, the server sends a ServerFinished message. This message contains an HMAC over the transcript. The client verifies this message to confirm that the server derived the same handshake secret.

2.4.5 Application Data

After the handshake succeeds, both parties derive application keys using HKDF. The client and server then exchange encrypted application messages using AES-GCM.

3 Security:

The application keys are derived from the X25519 shared secret and the password derived verifier. The X25519 shared secret is never transmitted over the network. A passive attacker can observe the public keys but cannot compute the shared secret without the private ephemeral keys. After the handshake, HKDF derives application keys from the handshake secret. Application data is encrypted using AES-GCM. Therefore, the plaintext application messages are protected from passive network attackers.

Handshake integrity is provided by the server signature and the HMAC values in ClientAuth and ServerFinished. These values are computed over the handshake transcript, so changes to handshake messages will cause verification to fail. Application data integrity is provided by AES-GCM. If an attacker modifies ciphertext or associated data, AES-GCM decryption fails.

The server has a certificate signed by the CA. The client first verifies the CA signature on the certificate using the trusted CA public key. If the certificate is valid, the client extracts the server public key from the certificate. The client then verifies the server's signature over the handshake transcript. This proves that the server owns the private signing key corresponding to the certified public key.

During registration, the server stores a salt and password derived verifier. The plaintext password is not stored. During the handshake, the client uses the password and salt to derive the same verifier. This value is combined with the X25519 shared secret to derive the handshake secret.

The client then sends an HMAC authenticator. The server computes the expected authenticator using its stored verifier. If the password is incorrect, the authenticator does not match, and the server rejects the handshake. This behavior was confirmed during testing. The correct password completed the handshake successfully, while an incorrect password caused the server to reject client authentication.

```
>> python client.py sai itsme
[Client] Loading CA public key...
[Client] Connecting to 127.0.0.1:4444...
[Client] Sent ClientHello.
[Client] Received ServerHello.
[Client] Certificate verified.
[Client] Server signature verified.
[Client] Sent ClientAuth.
[Client] ServerFinished verified.
[Client] Secure channel established.
[Client] Sent encrypted application message.
[Client] Decrypted server response: Hello client, your secure message was received.
```

Figure 1: Correct Password Scenario

```
(venv) (base) PS C:\Users\saith\ASU\Course_Materials\Fourth Semester\C
SE_539_Applied_Cryptography\TLS-PAKE-Project> python client.py sai wro
ngpassword
[Client] Loading CA public key...
[Client] Connecting to 127.0.0.1:4444...
[Client] Sent ClientHello.
[Client] Received ServerHello.
[Client] Certificate verified.
[Client] Server signature verified.
[Client] Sent ClientAuth.
[Client] Server error: Client authentication failed
```

Figure 2: Wrong Password Scenario

Each handshake uses fresh client and server nonces and fresh X25519 ephemeral keys. These values are included in the transcript and key derivation. As a result, the authenticator from one session is bound to that specific session and cannot be reused successfully in another session.

The protocol generates fresh X25519 ephemeral keys for each handshake. The application keys are derived from the ephemeral shared secret and transcript bound values. Since the ephemeral private keys are not reused or stored, past session keys cannot be recomputed from the server's long term signing key alone.

4 Implementation

The project is implemented in Python. The client and server communicate through TCP sockets over 127.0.0.1. JSON is used for protocol messages, and each message is sent with a 4 byte length prefix.

The `crypto_utils.py` file contains reusable cryptographic helper functions. It includes functions for Base64 encoding, canonical JSON encoding, SHA-256 hashing, HMAC-SHA256, Ed25519 signing and verification, X25519 key exchange, HKDF key derivation, and AES-GCM encryption and decryption.

The `messages.py` file implements message framing over TCP sockets. Since TCP does not preserve message boundaries, each JSON message is sent as: 4 byte message length || JSON message

The `storage.py` file provides helper functions for creating directories and reading or writing JSON files. It is used to store certificates and the password database.

The `ca.py` file emulates the Certificate Authority. It generates the CA key pair, generates the server signing key pair, creates a JSON server certificate, signs the certificate using the CA private key, and saves all generated files in the `certs/` directory.

The `setup.py` file is used for password registration. The user is registered using the command: *python setup.py register sai itsme*

The `server.py` file implements the server side handshake. It loads the server private key, server certificate, and password database. It then listens for a client connection on localhost. After receiving ClientHello, the server sends ServerHello with its certificate, ephemeral public key, password salt, and signature. The server then verifies the client's HMAC authenticator. If authentication succeeds, the server sends ServerFinished, derives application keys, decrypts the client's encrypted message, and sends an encrypted response.

The `client.py` file implements the client side handshake. It loads the CA public key, connects to the server, sends ClientHello, verifies the server certificate, verifies the server signature, derives the password based verifier, sends ClientAuth, verifies ServerFinished, derives application keys, and exchanges encrypted application data.

The `protocol.py` file stores protocol constants such as the host, port, HKDF labels, HMAC labels, and AES-GCM associated data labels. This keeps the main protocol files cleaner and reduces repeated string values.

5 Conclusion

This project implemented a simplified TLS 1.3 style handshake with PAKE inspired client authentication. The server is authenticated using a CA signed certificate and an Ed25519 signature. The client is authenticated using a password derived verifier and HMAC authenticator. After successful authentication, both parties derive application traffic keys using HKDF and exchange encrypted messages using AES-GCM.

The implementation was tested with both correct and incorrect passwords.

The server is started with: `python server.py`

The client was run with the correct password: `python client.py sai itsme`

The client was then run with an incorrect password: `python client.py sai wrongpassword`

The outputs received are as shown in *Figure 1* and *Figure 2*.

The implementation demonstrates the composition of several cryptographic primitives, including certificates, digital signatures, ephemeral key exchange, password based authentication, key derivation, transcript authentication, and authenticated encryption. The final result is a working client-server protocol that provides secrecy, integrity, and mutual authentication.